Multivariate Gaussians

A mean point and a covariance ellipse

Prof. Nipun Batra

Mathematical Foundations for AI · IIT Gandhinagar

Why multivariate Gaussians appear in ML

Common uses of Gaussian distributions in ML

You will read...	**Where**
"weights initialized from $\mathcal{N}(0, 0.02^2)$*"*	GPT-2's actual init
"add noise $\varepsilon \sim \mathcal{N}(0, I)$ *at every step"*	diffusion image generators
"sample a latent $z \sim \mathcal{N}(0, I)$*"*	variational autoencoders
"we assume Gaussian observation noise"	regression, Kalman filters, GPs

These applications use Gaussians for different reasons: initialization, injected noise, latent variables, and observation models.

Which mathematical properties make the Gaussian useful?

Three useful properties

1. **Limit behavior.** Standardized sums of many i.i.d. finite-variance effects approach a Gaussian. (The **Central Limit Theorem**; demonstrated today.)
2. **Maximum entropy under moment constraints.** Among densities with a specified mean and covariance, the Gaussian has maximum entropy. (Stated today, proved in Module 5.)
3. **Closure.** Marginals, conditionals, affine transforms, and sums of independent Gaussians remain Gaussian, with explicit parameter formulas.

These three properties explain many Gaussian models, but using a Gaussian remains a modeling assumption whose adequacy should be checked against the application.

L12 gave us bumps. But data is a vector.

Last lecture: densities for **one** continuous quantity — $p(x)$, area = probability, $\mathcal{N}(\mu, \sigma^2)$.

But since L3, our data has been **vectors**: (height, weight) · a 784-pixel image · an embedding · a parameter vector $\boldsymbol{\theta}$.

- Height and weight are not two separate stories: tall people tend to be heavier. A model of each alone **misses the relationship**.
- We need one density over $\mathbb{R}^d$ — a **joint** — that knows how coordinates co-vary.

Today's object: $x \sim \mathcal{N}(\mu, \Sigma)$ — a bump over $\mathbb{R}^d$.
One point μ says *where*; one matrix Σ says *what shape*.

Learning outcomes

By the end of this lecture you will be able to:

1. Write and **dissect the MVN density** — and say what every symbol is doing there.
2. Read a **covariance matrix** like a picture: variances, correlation, tilt.
3. Compute a 2×2 covariance **by hand** from raw data.
4. **Marginalize** (project) and **condition** (slice) a Gaussian — with formulas.
5. Score outliers with **Mahalanobis distance** — "how many σ's, direction-aware".
6. Prove the ⭐ fact: **contours are ellipses along the eigenvectors of Σ** (L5, cashed in).
7. Explain why Gaussians are everywhere: **CLT, max entropy, closure**.

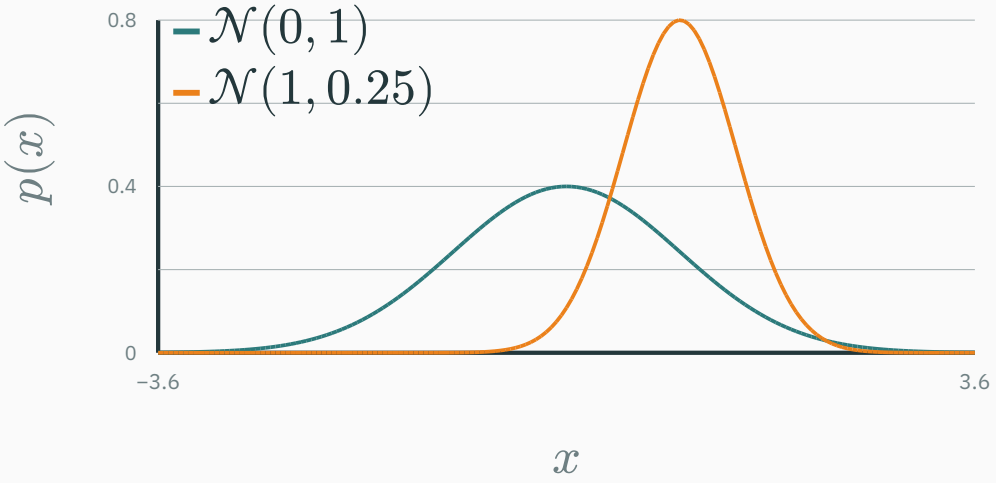
From one bump to a 2-D bump

The 1-D bell, recalled (L12)

V VISUAL

$$p(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

two numbers: center μ, spread σ



The parameters of $\mathcal{N}(\mu, \sigma^2)$ are (μ, σ). What replaces them when x becomes a **vector**?

First 2-D density: two independent bells, multiplied

Let x and y be **independent** Gaussians (L12: independence means the joint density factorizes):

$$p(x,y) = p(x)\cdot p(y) = \frac{1}{\sigma_1\sqrt{2\pi}} e^{-\frac{(x-\mu_1)^2}{2\sigma_1^2}} \times \frac{1}{\sigma_2\sqrt{2\pi}} e^{-\frac{(y-\mu_2)^2}{2\sigma_2^2}}$$

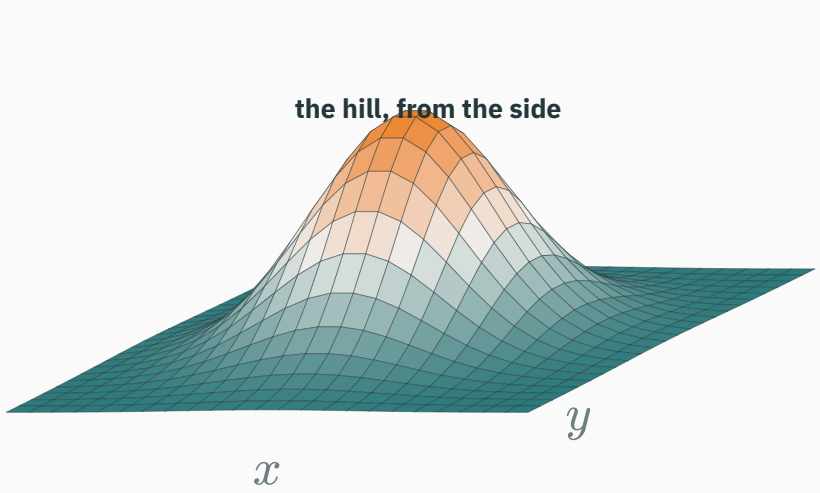
Collect the pieces — constants together, exponents added:

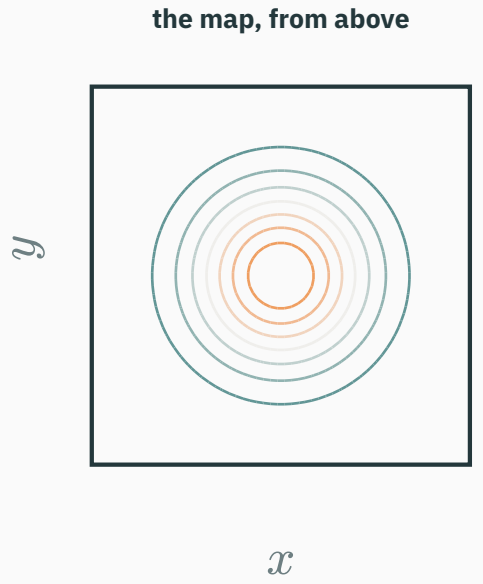
$$p(x,y) = \frac{1}{2\pi\,\sigma_1\sigma_2} \exp\left(-\frac{1}{2}\left[\frac{(x-\mu_1)^2}{\sigma_1^2} + \frac{(y-\mu_2)^2}{\sigma_2^2}\right]\right)$$

A perfectly legal density over $\mathbb{R}^2$ — our first 2-D bump. Every multivariate Gaussian is this, plus one twist we'll add shortly.

The 2-D bump, two views

V VISUAL





same object ($\mu_1 = \mu_2 = 0, \sigma_1 = \sigma_2 = 1$), sampled live from its formula — the right view's rings are **contours**: curves of equal density

Cartographers had this figured out long ago: a hill **is** its contour map. We will live in the map view.

Probability is volume now

L12's rule was "probability = **area** under the curve". One dimension up:

$$P((x, y) \in A) = \int\int_A p(x, y)\, dx\, dy$$

probability = **volume** under the hill, above the region A

- Total volume under the whole hill: exactly 1.
- $p(x, y)$ is still a **density**, not a probability: P of any single point is 0, and p can exceed 1 (both L12 facts, still in force).

Contours are the map's way of drawing the hill. Everything today — shape, slicing, projecting — will be read off contour pictures.

Repackage the exponent: a quadratic form appears

Write $\boldsymbol{x} = \begin{pmatrix} x \\ y \end{pmatrix}$, $\boldsymbol{\mu} = \begin{pmatrix} \mu_1 \\ \mu_2 \end{pmatrix}$, $\boldsymbol{d} = \boldsymbol{x} - \boldsymbol{\mu}$. The exponent was a sum of squares — watch it become a **quadratic form** (L9!):

$$\frac{(x-\mu_1)^2}{\sigma_1^2} + \frac{(y-\mu_2)^2}{\sigma_2^2} = \boldsymbol{d}^\top \begin{pmatrix} 1/\sigma_1^2 & 0 \\ 0 & 1/\sigma_2^2 \end{pmatrix} \boldsymbol{d} = \boldsymbol{d}^\top \Sigma^{-1} \boldsymbol{d}$$

where $\Sigma = \begin{pmatrix} \sigma_1^2 & 0 \\ 0 & \sigma_2^2 \end{pmatrix}$ collects the two variances — our first covariance matrix, and Σ^{-1} simply divides each squared deviation by its own variance.

L9 planted exactly this: *"the Gaussian's exponent IS a quadratic form, with $A = \Sigma^{-1}$."* The IOU starts being paid right here.

Repackage the constant: a determinant appears

The normalizing constant can be written with a determinant — for a diagonal matrix, $\det$ is the product of the diagonal entries (L4):

$$\frac{1}{2\pi\,\sigma_1\sigma_2} = \frac{1}{2\pi\,\sqrt{\sigma_1^2\,\sigma_2^2}} = \frac{1}{2\pi\,\det(\Sigma)^{1/2}}$$

$$p(\boldsymbol{x}) = \tfrac{1}{2\pi\,\det(\Sigma)^{1/2}}\,\exp\!\left(-\tfrac{1}{2}\boldsymbol{d}^\top\Sigma^{-1}\boldsymbol{d}\right)$$

nothing was added — the independent case re-packaged itself.

The general formula: keep the packaging, free the matrix

The repackaged formula makes sense for **any** symmetric positive-definite Σ — not just diagonal ones. That generalization **is** the multivariate Gaussian, in any dimension d:

$$p(\boldsymbol{x}) = \frac{1}{(2\pi)^{d/2}\det(\Sigma)^{1/2}} \exp\left(-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^\top \Sigma^{-1}(\boldsymbol{x}-\boldsymbol{\mu})\right), \qquad \boldsymbol{x} \sim \mathcal{N}(\boldsymbol{\mu}, \Sigma)$$

- $\boldsymbol{\mu} \in \mathbb{R}^d$: the **mean vector** — where the peak sits.
- $\Sigma \in \mathbb{R}^{d\times d}$: the **covariance matrix** — symmetric, positive definite. Off-diagonal entries are the “twist”: they **tilt** the bump, encoding co-variation that independence forbade.

This formula is the independent case rewritten in matrix notation, then extended from diagonal covariance to any symmetric positive-definite covariance. The next sections interpret each term geometrically.

The formula, dissected

$$p(\boldsymbol{x}) = \underbrace{\frac{1}{(2\pi)^{d/2}\det(\Sigma)^{1/2}}}_{\text{volume bookkeeping}} \exp\left(-\frac{1}{2}\underbrace{(\boldsymbol{x}-\boldsymbol{\mu})^\top \Sigma^{-1}(\boldsymbol{x}-\boldsymbol{\mu})}_{\text{a quadratic form in } \boldsymbol{x}-\boldsymbol{\mu}}\right)$$

- The **exponent** is L9's quadratic form $\boldsymbol{d}^\top A\boldsymbol{d}$ with $A = \Sigma^{-1}$ — L9 planted exactly this: *"the bowl's ellipse contours are a Gaussian's, with $A = \Sigma^{-1}$"*. Today that IOU is paid.
- The **normalizer** only rescales height so total volume is 1 — $\det(\Sigma)^{1/2}$ is the bump's footprint volume (L4: determinants measure volume). It never affects the shape.
- **Sanity check** $d = 1$, $\Sigma = (\sigma^2)$: $\det(\Sigma)^{1/2} = \sigma$, $\Sigma^{-1} = 1/\sigma^2$ — L12's bell, exactly.

Reading the covariance matrix

Σ

The covariance matrix, defined

$$\Sigma_{ij} = \mathbb{E}\big[(X_i - \mu_i)(X_j - \mu_j)\big]$$

- **Diagonal** ($i = j$): $\Sigma_{ii} = \mathbb{E}\big[(X_i - \mu_i)^2\big] = \mathrm{Var}(X_i)$ — each coordinate's own spread.
- **Off-diagonal** ($i \neq j$): do X_i and X_j move **together**? Positive: above-average together. Negative: one up, the other down. Zero: no linear relationship.
- Swap i and j: same product, same expectation. So $\Sigma_{ij} = \Sigma_{ji}$ — **every covariance matrix is symmetric**, automatically. (You know what that buys us: L5′s spectral theorem. Hold that thought.)

Numbers: four data points, by hand

Four measurements of (x, y): $(1,5), (3,3), (3,5), (5,7)$. Mean: $\boldsymbol{\mu} = \begin{pmatrix} 3 \\ 5 \end{pmatrix}$. Deviations $\boldsymbol{d_i} = \boldsymbol{x_i - \mu}$, then average the products:

point	**deviation** (d_x, d_y)	d_x^2	d_y^2	$d_x d_y$
$(1,5)$	$(-2,0)$	4	0	0
$(3,3)$	$(0,-2)$	0	4	0
$(3,5)$	$(0,0)$	0	0	0
$(5,7)$	$(2,2)$	4	4	4
mean		$\mathbf{2}$	$\mathbf{2}$	$\mathbf{1}$

$$\Sigma = \begin{pmatrix} \text{mean } d_x^2 & \text{mean } d_x d_y \\ \text{mean } d_x d_y & \text{mean } d_y^2 \end{pmatrix} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$$

the slide recomputes this Σ from the raw points at compile time (chalkdust) ✓

Wait — you have met this matrix before

$$\Sigma = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$$

L5's running matrix. Its eigenvectors are $(1, 1)$ and $(1, -1)$, with eigenvalues 3 and 1. The contour derivation later in the lecture will turn these into ellipse axes and squared axis scales.

For any direction $\boldsymbol{a}$, the variance of the projection $\boldsymbol{a}^\top \boldsymbol{x}$ is

$$\mathrm{Var}(\boldsymbol{a}^\top \boldsymbol{x}) = \boldsymbol{a}^\top \Sigma\, \boldsymbol{a}$$ — a quadratic form again, and variances can't be negative...

$a^\top \Sigma\, a \geq 0$ for every a: covariance matrices are exactly L9's positive semi-definite club. Σ is a machine that turns directions into variances.

Correlation: the unit-free reading

Covariance carries units (cm·kg — unreadable). Normalize by both spreads:

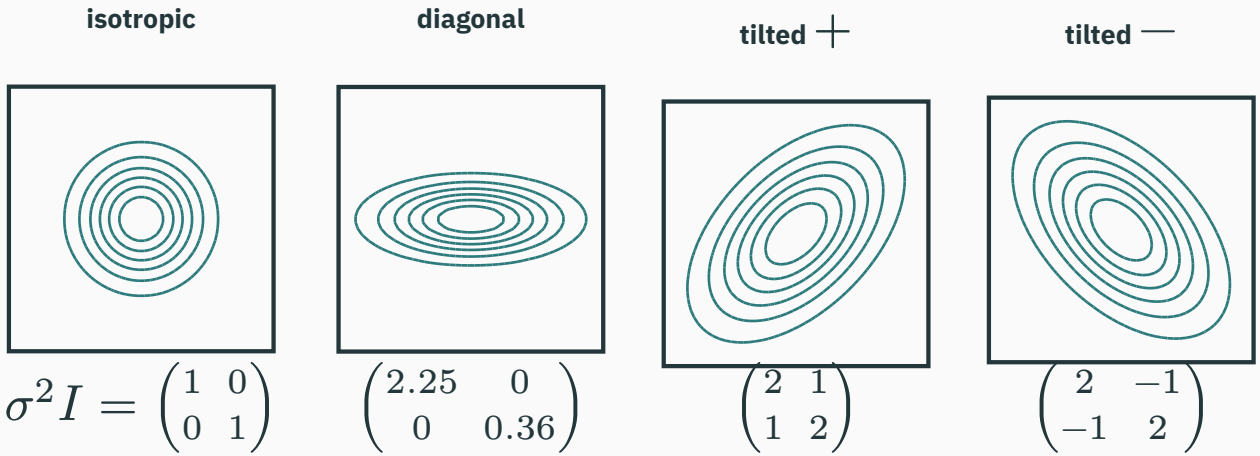
$$\rho = \frac{\Sigma_{12}}{\sigma_1 \sigma_2} = \frac{\Sigma_{12}}{\sqrt{\Sigma_{11}\Sigma_{22}}}, \qquad \rho \in [-1, 1]$$

- $\rho = \pm 1$: a perfect line · $\rho = 0$: no **linear** co-movement · sign = tilt direction.
- Our running Σ: $\rho = \frac{1}{\sqrt{2 \cdot 2}} = 0.5$ — moderately, positively coupled.

For **Gaussians** (and only for them, in general): $\rho = 0 \Leftrightarrow$ independent. For arbitrary distributions, $\rho = 0$ does NOT imply independence — a caution flag worth remembering.

The gallery: what Σ does to the bump

V VISUAL

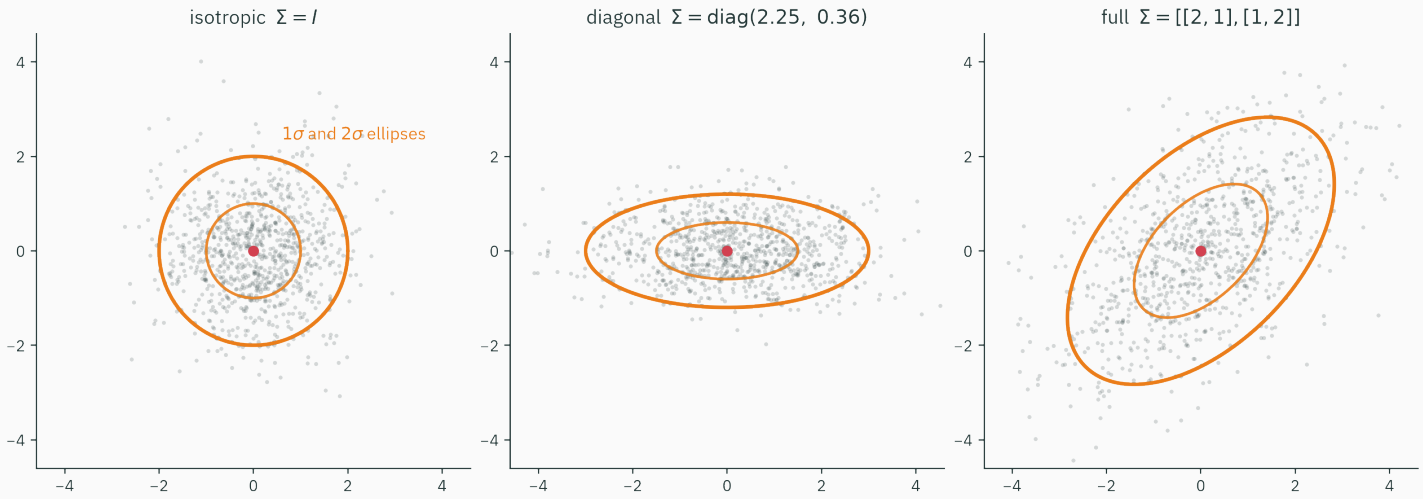


four live densities, one formula — only Σ changed: circles $\rightarrow$ axis-aligned stretch $\rightarrow$ tilt (sign of Σ_{12} picks the tilt's direction)

μ moves the bump. The diagonal of Σ stretches it. The off-diagonal tilts it.

Sampled clouds obey the same Σ

V VISUAL



900 samples from each density; the 1σ and 2σ ellipses are drawn from Σ **alone**. The cloud fills its ellipse — simulation and formula agree.

The picture to keep: **a Gaussian is a data cloud's idealization** — center μ, shape Σ. Where the ellipse comes from: the ⭐ derivation, soon.

Interactive: bend the ellipse yourself

I INTERACTIVE

INTERACTIVE

↗ nipunbatra.github.io/interactive-articles/multivariate-normal

Drag the entries of Σ (variances and correlation) and watch three things respond at once: the sampled **cloud**, the **contour ellipse**, and the **marginals** on the walls. Then slide the conditioning bar and watch a slice move — the two operations of the next section, live under your mouse.

Five minutes with the sliders builds the reflex: you should **see** $\begin{pmatrix} 4 & -3 \\ -3 & 9 \end{pmatrix}$ as a shape before you compute anything about it.

Code: Σ in one line

```
X = np.array([[1., 5.], [3., 3.], [3., 5.], [5., 7.]])
X.mean(axis=0)              # array([3., 5.])          = μ
np.cov(X.T, bias=True)      # [[2., 1.], [1., 2.]]     = Σ  (divide by n)
np.cov(X.T)                 # [[2.67, 1.33], ...]      (divide by n-1)
```

```
d = torch.distributions.MultivariateNormal(
        loc=torch.tensor([3., 5.]),
        covariance_matrix=torch.tensor([[2., 1.], [1., 2.]]))
xs = d.sample((100_000,))   # 100k points from our Gaussian
torch.cov(xs.T)             # ≈ [[2.00, 1.00], [1.00, 2.00]] ✓
```

`bias=True` divides by n (what we did by hand); the default divides by $n-1$. L14 explains the estimation distinction between these conventions.

Checkpoint: read the matrix

Q QUESTION

$\Sigma = \begin{pmatrix} 4 & -3 \\ -3 & 9 \end{pmatrix}$ **describes a data cloud. Which reading is correct?**

A. $\sigma_x = 4$, $\sigma_y = 9$, cloud tilted up-right

B. $\sigma_x = 2$, $\sigma_y = 3$, cloud tilted down-right

C. Invalid — a covariance matrix can never contain negative entries

D. x and y are independent, since Σ is symmetric

Answer: read the matrix

A ANSWER

Correct: B — $\sigma_x = 2$, $\sigma_y = 3$, tilted down-right

Why: The **diagonal** holds variances, so $\sigma_x = \sqrt{4}$, $\sigma_y = \sqrt{9}$ — A read them as standard deviations. $\rho = -3/(2 \cdot 3) = -0.5 < 0$: as x rises, y tends to fall — a down-right tilt. C: only the *diagonal* must be non-negative; off-diagonals may be negative freely (validity is positive semi-definiteness: here $\det = 36 - 9 = 27 > 0$, trace > 0 — L9's hand test says PD ✓). D: independence needs $\Sigma_{12} = 0$; symmetry is free.

Slice and project: marginals and conditionals

Two questions you ask every joint

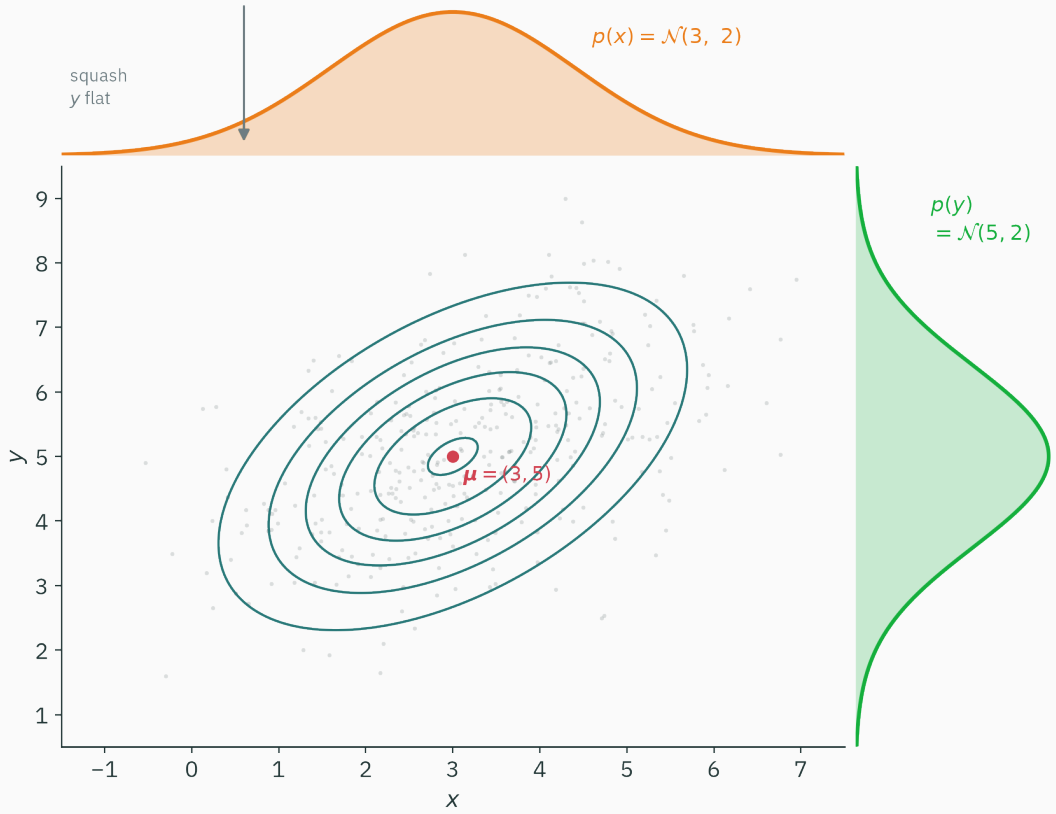
Your model is $p(x, y)$ over (height, weight). Two everyday questions:

1. *"What's the distribution of weight — ignoring height entirely?"*
   → the **marginal** $p(y) = \int p(x, y)\, dx$: sum away what you don't care about.
2. *"What's the distribution of weight **among 180 cm people**?"*
   → the **conditional** $p(y \mid x = 180) = p(180, y)/p(180)$: freeze what you know, renormalize.

Geometrically, marginalization projects the joint density and conditioning takes and renormalizes a slice. The next two figures show both operations.

Project = marginalize

V VISUAL



Squash the hill onto a wall — every y collapses into its x column.

For our worked Gaussian:

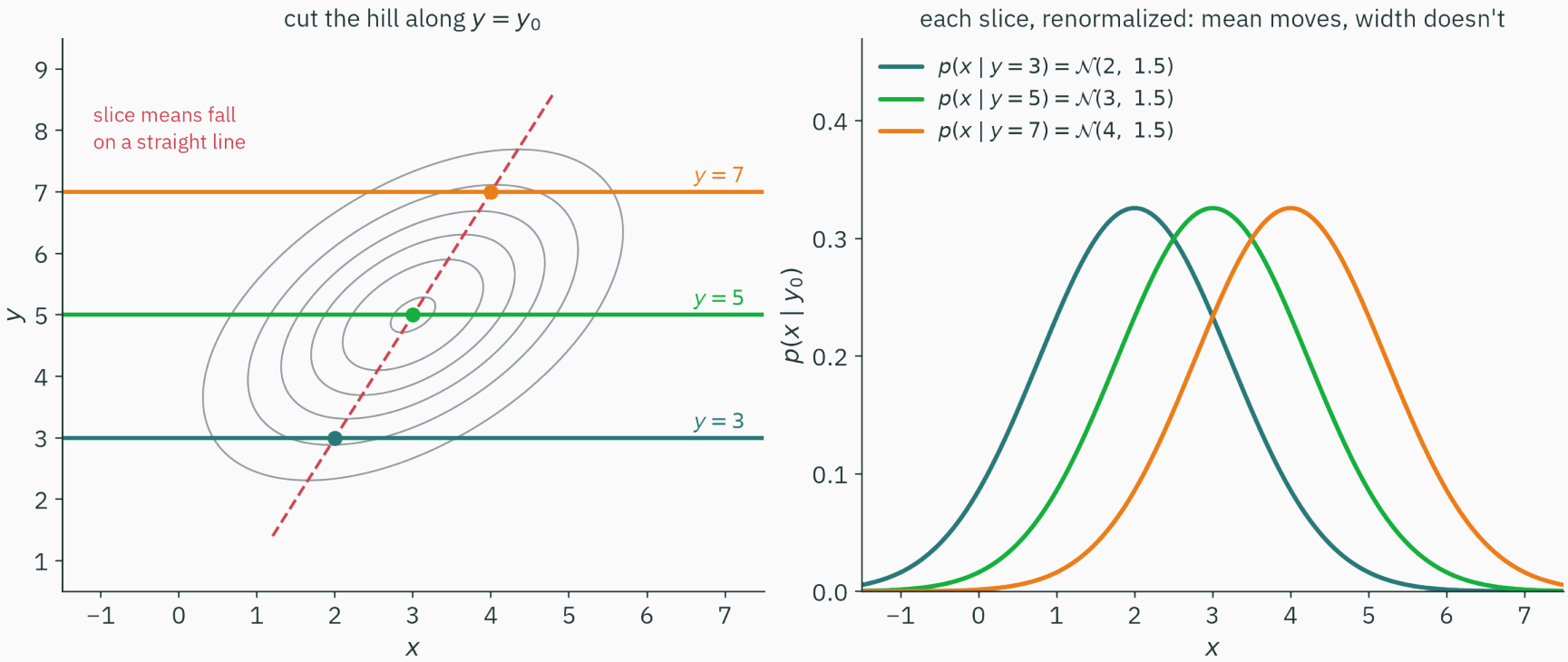
$$p(x) = \mathcal{N}(3, 2)$$

$$p(y) = \mathcal{N}(5, 2)$$

Marginals: read them off μ and Σ — keep your entries, delete the rest. No integral needed.

Slice = condition

V VISUAL



Cut along $y = y_0$: the profile revealed is $p(x, y_0)$ — a bump-shaped sliver.
Renormalize (divide by $p(y_0)$) and it becomes a legal 1-D density: $p(x \mid y = y_0)$.

The slice, quantified

Both panels of that figure obey two formulas (for 2-D; general d: same shapes, block form):

$$\mu_{x|y} = \mu_x + \frac{\Sigma_{12}}{\Sigma_{22}}(y_0 - \mu_y) \qquad \sigma^2_{x|y} = \Sigma_{11} - \frac{\Sigma^2_{12}}{\Sigma_{22}}$$

Our numbers ($\boldsymbol{\mu} = (3, 5)$, $\Sigma_{12} = 1$, $\Sigma_{22} = 2$): observe $y_0 = 7$ →

$$\mu_{x|7} = 3 + \frac{1}{2}(7 - 5) = 4 \qquad \sigma^2_{x|y} = 2 - \frac{1}{2} = 1.5 \quad \text{(matches the figure's orange slice)}$$

- The mean **shifts linearly** with y_0 — the dashed line through the slice means is a preview of **regression** (L14's linear regression lives inside this picture).
- The variance **shrank** ($1.5 < 2$) and doesn't depend on y_0 at all — knowing y helps by the same amount everywhere. A distinctly Gaussian privilege.

Closure: every answer stayed in the family

operation on $\mathcal{N}(\mu, \Sigma)$	**result**
marginalize (project)	Gaussian — keep the relevant entries of μ, Σ
condition (slice)	Gaussian — mean shifts linearly, variance shrinks
linear map $Ax + b$	Gaussian — $\mathcal{N}(A\mu + b, A\Sigma A^\top)$
sum of independent Gaussians	Gaussian — means add, covariances add

The Gaussian family is closed under common linear operations used in ML, each with an explicit formula.

This closure is why **Kalman filters** track rockets in real time and **Gaussian processes** fit in a page of code: every update is "Gaussian in, Gaussian out". General block formulas: MML §6.5.

Code: check the slice by filtering samples

Don't trust the formula? Condition **empirically** — keep only samples where $y \approx 7$:

```
xs = d.sample((400_000,)).numpy()          # our N([3,5], [[2,1],[1,2]])
band = xs[np.abs(xs[:, 1] - 7.0) < 0.05]   # the thin slice y ≈ 7
band[:, 0].mean(), band[:, 0].var()
# (4.003, 1.497)     ← formula predicted N(4, 1.5) ✓
```

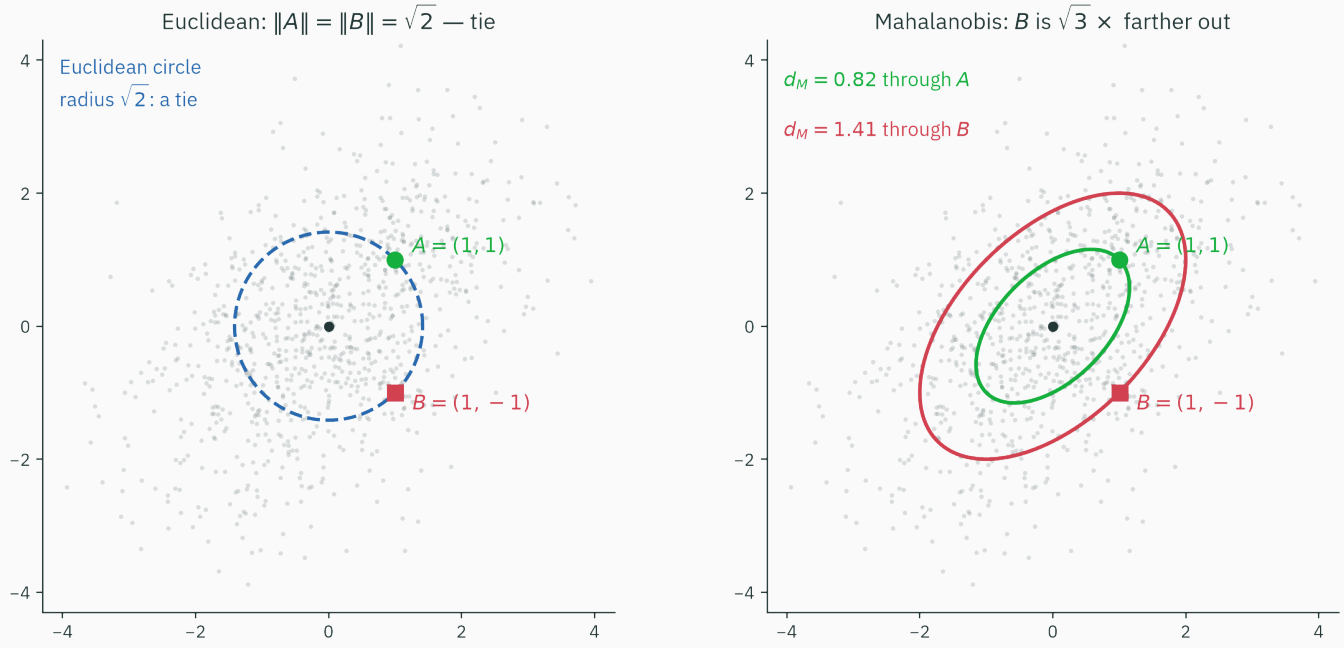
The filtered histogram is the renormalized slice; simulation agrees with the conditional-density formula.

Marginal = project. Conditional = slice. Both stay Gaussian — that's the spine of this lecture.

Mahalanobis distance: direction-aware σ's

Two points, same distance, different stories

V VISUAL



$A = (1, 1)$ and $B = (1, -1)$ sit at **identical** Euclidean distance $\sqrt{2}$ from the mean — yet the cloud disagrees violently: A is in the thick of it, B is out in the cold.

Euclidean distance is **shape-blind**. We need a ruler that has seen Σ.

In 1-D, you already own the fix

L12's **z-score**: don't ask "how far?", ask **"how many standard deviations?"**

$$z = \frac{x - \mu}{\sigma}$$ 3 cm from the mean means nothing until you know σ

In d dimensions the question sharpens: standard deviations **in which direction?** Along the cloud's long axis, spread is large — a big step is cheap. Across it, spread is small — the same step is expensive.

We need a distance that rescales every direction by that direction's own spread. Σ knows every direction's spread — so the ruler must be built from Σ.

Mahalanobis distance, defined

$$d_M^2(\boldsymbol{x}) = (\boldsymbol{x} - \boldsymbol{\mu})^\top \Sigma^{-1} (\boldsymbol{x} - \boldsymbol{\mu})$$

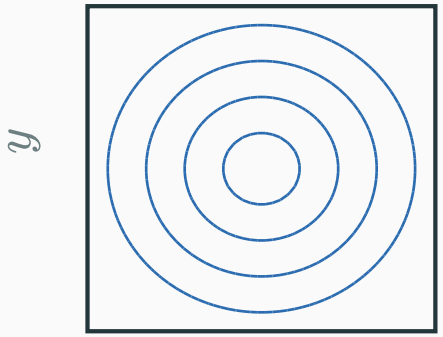
- Why Σ^{-1}: dividing by variance, matrix-style — big-spread directions get **discounted**, tight directions get **amplified**.
- Sanity, $d = 1$: $d_M^2 = (x - \mu)^2 / \sigma^2$ — the z-score squared, exactly.
- Now look again at the Gaussian: $p(\boldsymbol{x}) \propto \exp\left(-\frac{1}{2} d_M^2(\boldsymbol{x})\right)$. **The exponent we dissected IS Mahalanobis distance.**

Equal density $\Leftrightarrow$ equal Mahalanobis distance.
A Gaussian's contours are Mahalanobis "circles".

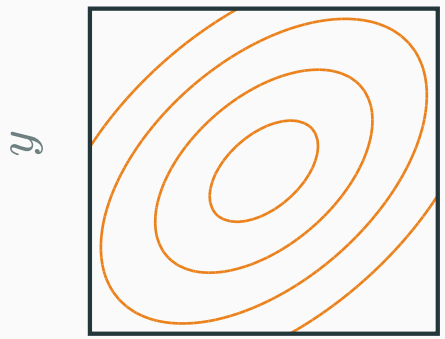
One ruler, two geometries

V VISUAL

Euclidean $d^\top d$: circles



Mahalanobis $d^\top \Sigma^{-1} d$: ellipses



same four level values on both plots, $\Sigma = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$ — the Mahalanobis unit ball is the covariance ellipse itself

Compute the conditional distribution

The slide inverts Σ live (chalkdust `la.inv`):

$$\Sigma^{-1} = \begin{pmatrix} 0.6667 & -0.3333 \\ -0.3333 & 0.6667 \end{pmatrix} = \frac{1}{3}\begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix}$$

Score both suspects — hand-checkable in two lines each:

$$d_M^2(A) = (1,1)\,\Sigma^{-1}\begin{pmatrix}1\\1\end{pmatrix} = 0.6667 \quad \Rightarrow \quad d_M(A) = 0.82 \;\; \text{‘standard deviations’}$$

$$d_M^2(B) = (1,-1)\,\Sigma^{-1}\begin{pmatrix}1\\-1\end{pmatrix} = 2.0000 \quad \Rightarrow \quad d_M(B) = 1.41$$

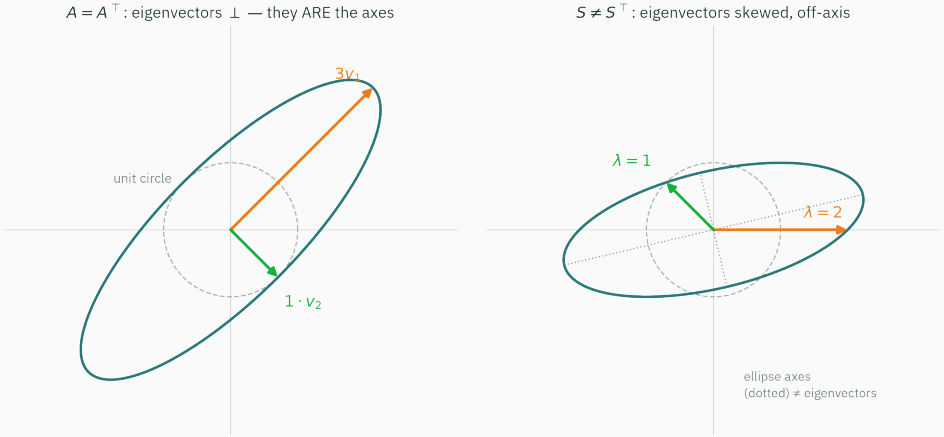
Why exactly those numbers? A lies along $(1,1)$ — the direction with variance 3. B lies along $(1,-1)$ — variance 1. Eigen-directions again… which brings us to the ⭐.

⭐ The ellipse is the eigendecomposition of Σ

The claim — and where you first saw it

V VISUAL

Claim. The contours of $\mathcal{N}(\mu, \Sigma)$ are ellipses centered at μ, with axes along the **eigenvectors of Σ** and semi-axes proportional to $\sqrt{\lambda_i}$.



L5 anticipated the claim: a Gaussian data cloud's covariance eigenvectors give the axes of its density ellipses. The next three slides derive it.

⭐ Step 1: a contour is a level set of the exponent

D DERIVATION

Fix a density value c and take $\log$ of both sides of $p(\boldsymbol{x}) = c$ — the normalizer is a constant, so it folds into the right-hand side:

$$p(\boldsymbol{x}) = c \quad \Leftrightarrow \quad (\boldsymbol{x} - \boldsymbol{\mu})^\top \Sigma^{-1} (\boldsymbol{x} - \boldsymbol{\mu}) = r^2 \quad (\text{some constant } r^2 \geq 0)$$

($\exp$ is strictly monotone, so no information is lost taking the $\log$ — L2′s log-space move, working geometry duty.)

Contours of the Gaussian = level sets of the quadratic form $d^\top A\, d$ with $A = \Sigma^{-1}$ — precisely L9′s bowl. What remains: which ellipses, pointed where?

⭐ Step 2: Σ and Σ^{-1} share eigenvectors

D DERIVATION

Let $\Sigma q = \lambda q$ (spectral theorem, L5: Σ symmetric → orthonormal q_i, real λ_i; positive definite → all $\lambda_i > 0$, so Σ^{-1} exists). Multiply both sides by Σ^{-1} and divide by λ:

$$\Sigma q = \lambda q \quad \Rightarrow \quad q = \lambda\, \Sigma^{-1} q \quad \Rightarrow \quad \Sigma^{-1} q = \frac{1}{\lambda}\, q$$

Same eigenvectors, reciprocal eigenvalues. The bowl $d^\top \Sigma^{-1} d$ and the covariance Σ agree about the special directions — they only disagree about which one is "long".

Where Σ has a **large** λ (much spread), Σ^{-1} has a **small** $1/\lambda$ (cheap distance) — the discount from the Mahalanobis slide, now with a mechanism.

⭐ Step 3: switch to the eigen-basis

D DERIVATION

L5/L9′s signature move. Write $\Sigma = Q\Lambda Q^\top$, change coordinates $\boldsymbol{t} = Q^\top(\boldsymbol{x} - \boldsymbol{\mu})$ — a rotation onto the eigen-axes. The cross-terms die:

$$(\boldsymbol{x} - \boldsymbol{\mu})^\top \Sigma^{-1} (\boldsymbol{x} - \boldsymbol{\mu}) = \boldsymbol{t}^\top \Lambda^{-1} \boldsymbol{t} = \frac{t_1^2}{\lambda_1} + \frac{t_2^2}{\lambda_2}$$

So the contour $= r^2$ reads, in eigen-coordinates:

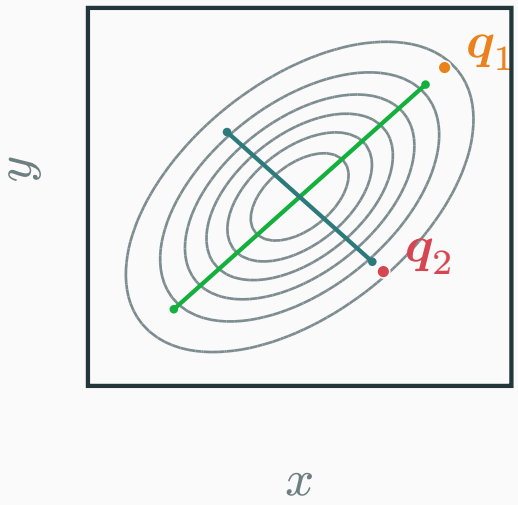
$$\frac{t_1^2}{r^2\lambda_1} + \frac{t_2^2}{r^2\lambda_2} = 1 \quad \text{— the standard ellipse equation, semi-axes } r\sqrt{\lambda_1} \text{ and } r\sqrt{\lambda_2}$$

Contours of $\mathcal{N}(\mu, \Sigma)$: ellipses centered at μ, axes along the eigenvectors q_i of Σ, semi-axes $\propto \sqrt{\lambda_i}$. ■

L5 planted it, L9 armed it, L13 proved it.

The theorem, checked live

V VISUAL



`la.eig-sym(Σ)`, run by this slide:

$$\lambda_1 = 3, \quad \boldsymbol{q}_1 = \begin{pmatrix} 0.707 \\ 0.707 \end{pmatrix}$$

$$\lambda_2 = 1, \quad \boldsymbol{q}_2 = \begin{pmatrix} 0.707 \\ -0.707 \end{pmatrix}$$

Axis half-length ratio:

$$\sqrt{\lambda_1} : \sqrt{\lambda_2} = \sqrt{3} : 1 \approx 1.73 : 1$$

contours sampled from the density; axis segments drawn from `eig-sym`'s output, lengths $\propto \sqrt{\lambda_i}$ — computed, not sketched

Where this one fact keeps cashing out

- **PCA (L6), demystified**: PCA's principal directions are the eigenvectors of the data covariance — i.e., **the axes of this ellipse**. L6 found them by SVD; today you know what they mean probabilistically.
- **Whitening**: rotate by $Q^\top$, divide by $\sqrt{\lambda_i}$ — any Gaussian becomes $\mathcal{N}(0, I)$. Data preprocessing = ellipse → circle.
- **Optimization preview (L17)**: quadratic loss surfaces have these same elliptical contours; the ratio $\lambda_{\max}/\lambda_{\min}$ (how **stretched** the ellipse is) will decide how fast gradient descent can go.

One picture — eigenvectors of a symmetric matrix as ellipse axes — now runs statistics (today), compression (L6), and optimization (L17).

Checkpoint: the long axis

Q QUESTION

$\mathcal{N}(\mu, \Sigma)$ with $\Sigma = \begin{pmatrix} 3 & 2 \\ 2 & 3 \end{pmatrix}$ has elliptical contours. Their long axis points along…

A. $(1, 0)$ — the first diagonal entry is largest

B. $(1, 1)$

C. $(1, -1)$

D. Can't say without knowing μ

Answer: the long axis

A ANSWER

Correct: B — along $(1, 1)$

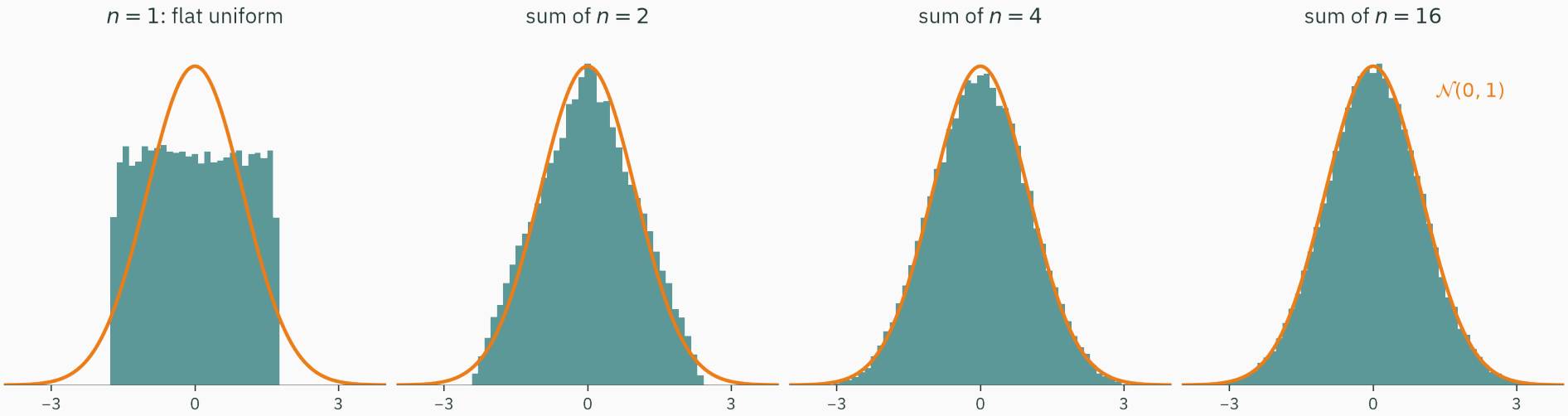
Why: Same drill as our running matrix: for $\begin{pmatrix} 3 & 2 \\ 2 & 3 \end{pmatrix}$, trace $= 6$ and $\det = 5$ give $\lambda = 5, 1$ (L5′s audit), with $(1, 1)$ the $\lambda = 5$ direction: most variance, longest axis ($\sqrt{5} : 1 \approx 2.2 : 1$). A: diagonal entries are per-**coordinate** variances — equal here (3 and 3), yet the ellipse is far from round; the tilt lives in the off-diagonal. D: μ only **slides** the ellipse; orientation is Σ's alone.

Why the Gaussian is everywhere

Sums of independent variables

V VISUAL

Take the **least** Gaussian thing available — a flat uniform — and add independent copies:



By $n = 4$ the bell shape is visible; by $n = 16$ the histogram closely follows $\mathcal{N}(0, 1)$ at the plotted resolution. Other i.i.d. finite-variance summands show the same standardized limit behavior.

The Central Limit Theorem, stated

Theorem (CLT, informal). If $X_1, \ldots, X_n$ are i.i.d. with mean μ and finite variance σ^2, then the standardized sum

$$\frac{X_1 + \cdots + X_n - n\mu}{\sigma\sqrt{n}} \to \quad \mathcal{N}(0,1) \quad \text{as } n \to \infty$$

- After centering and scaling, the limiting distribution depends on the summands through their mean and variance; the finite-n approximation quality still depends on the original distribution.
- Sensor error, measurement jitter, and aggregate effects often combine many small independent contributions, so the central limit theorem motivates Gaussian noise models.

The CLT is stated and illustrated here but not proved; a proof uses tools such as characteristic functions. See MML §6.4 for pointers.

The Gaussian maximizes entropy at fixed variance

Fact (maximum entropy). Among **all** densities on $\mathbb{R}$ with a given mean μ and variance σ^2, the Gaussian has the largest **entropy** — it is the least committed, most spread-out choice consistent with those two facts.

- If a modeling principle retains only $\boldsymbol{\mu}$ and Σ and otherwise maximizes entropy, it selects $\mathcal{N}(\boldsymbol{\mu}, \Sigma)$. This does not make every dataset Gaussian; it states the optimization criterion used to choose the model.
- "Entropy" — a number measuring non-commitment? Defining it properly is the whole business of **Module 5** (L22). This fact is planted today and **proved there**.

The CLT motivates Gaussian approximations for standardized sums; the maximum-entropy theorem selects a Gaussian when only mean and covariance are constrained.

Summary of the Gaussian modelling assumptions

Why Gaussian models recur in ML:

property	modelling consequence	status
CLT	some aggregated effects are approximately Gaussian after standardization	demoed today
max entropy	maximum-entropy density given μ, Σ	stated → L22
closure	marginals, conditionals, sums: **closed forms**	seen today

GPT-2's initialization, diffusion noise, VAE latents, and Kalman filters use different subsets of these properties.

Gaussian models combine tractable linear algebra with useful limit and maximum-entropy characterizations.

Summary

Lecture 13 — summary

- **The MVN**: a bump on $\mathbb{R}^d$ with density $\propto \exp\!\left(-\tfrac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^\top \Sigma^{-1}(\boldsymbol{x}-\boldsymbol{\mu})\right)$ — the exponent is L9′s quadratic form, $A = \Sigma^{-1}$.
- **Reading Σ**: diagonal = variances, off-diagonal = co-movement ($\rho = \Sigma_{12}/\sigma_1\sigma_2$); PSD because $\boldsymbol{a}^\top \Sigma \boldsymbol{a} = \mathrm{Var}(\boldsymbol{a}^\top \boldsymbol{x}) \geq 0$.
- **Marginal = project, conditional = slice** — both stay Gaussian; conditional mean moves linearly, variance shrinks by a constant.
- **Mahalanobis** $d_M^2 = (\boldsymbol{x}-\boldsymbol{\mu})^\top \Sigma^{-1}(\boldsymbol{x}-\boldsymbol{\mu})$: “how many σ’s, direction-aware”; equal density $\iff$ equal d_M.
- ⭐ **The ellipse**: contour axes = eigenvectors of Σ, semi-axes $\propto \sqrt{\lambda_i}$ (L5, paid).
- **Why everywhere**: CLT + max entropy (→ L22) + closure.

Read before L14 — MML §6.5–6.6 · CS229 probability notes handout. **Tutorial 6** this week: covariances by hand, slicing/projecting, the ellipse in NumPy.

Practice problems

Try on paper; check with NumPy (`np.cov`, `np.linalg.eigh`, `np.linalg.inv`).

1. Read $\Sigma = \begin{pmatrix} 9 & -5 \\ -5 & 4 \end{pmatrix}$: σ_x, σ_y, ρ, tilt direction. Then verify it **is** a valid covariance (L9 hand test: $\det$, trace).
2. Why can no dataset ever have covariance $\begin{pmatrix} 1 & 2 \\ 2 & 1 \end{pmatrix}$? (Hint: L9 classified this exact matrix. Find a direction $\boldsymbol{a}$ with "negative variance".)
3. Compute the covariance of $(0, 0), (2, 1), (4, 2)$ by hand. What is $\det(\Sigma)$? What has the "ellipse" collapsed into, and which L4 concept is showing up?
4. For our running $\Sigma = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$: compute d_M^2 of $(2, 2)$ and of $(-1, 1)$. Which is the outlier? Reconcile with the eigenvalues.
5. Sketch the 1-contour of $\mathcal{N}\left(\mathbf{0}, \begin{pmatrix} 5 & 3 \\ 3 & 5 \end{pmatrix}\right)$: axis directions, axis-length ratio. (Trace/det shortcut welcome.)
6. For $\boldsymbol{\mu} = (3, 5)$, $\Sigma = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$: write $p(y \mid x = 1)$, then verify by sampling + filtering in code (as we did for $y = 7$).

⭐⭐⭐ Sampling: every Gaussian is a warped $\mathcal{N}(0, I)$

★ OPTIONAL

L12 taught the machine to make 1-D standard normals. Stack d of them: $\boldsymbol{z} \sim \mathcal{N}(0, I)$ — a round cloud. How do we get $\mathcal{N}(\boldsymbol{\mu}, \Sigma)$ from it?

Apply a linear map and use **closure**: $\boldsymbol{x} = \boldsymbol{\mu} + A\boldsymbol{z}$ is Gaussian with mean $\boldsymbol{\mu}$ and covariance

$$\mathrm{Cov}(A\boldsymbol{z}) = A\,\mathrm{Cov}(\boldsymbol{z})\,A^\top = AA^\top$$

so we need a matrix with $AA^\top = \Sigma$ — a "square root" of Σ

The workhorse choice: the **Cholesky factor** L — lower-triangular, $LL^\top = \Sigma$, exists for every positive-definite Σ, cheap to compute.

$$\boldsymbol{z} \sim \mathcal{N}(0, I), \quad \boldsymbol{x} = \boldsymbol{\mu} + L\boldsymbol{z} \quad \Rightarrow \quad \boldsymbol{x} \sim \mathcal{N}(\boldsymbol{\mu}, \Sigma)$$

⭐⭐⭐ Cholesky in code — and in pictures

★ OPTIONAL

```
Sigma = np.array([[2., 1.], [1., 2.]])
L = np.linalg.cholesky(Sigma)           # L @ L.T == Sigma
z = rng.standard_normal((100_000, 2))   # round cloud  N(0, I)
x = np.array([3., 5.]) + z @ L.T        # our Gaussian N(μ, Σ)
np.cov(x.T)                             # ≈ [[2.00, 1.00], [1.00, 2.00]] ✓
```

- Geometry: L maps the unit circle of z onto **the covariance ellipse**; the same linear map is also a Gaussian sampling construction.
- Another valid square root, straight from the ⭐: $\Sigma^{1/2} = Q\Lambda^{1/2}Q^\top$ — stretch by $\sqrt{\lambda_i}$ along the eigen-axes, rotate. Different A, same $AA^\top$, same distribution.
- This is precisely how `torch.distributions.MultivariateNormal` samples (it stores `scale_tril` — the Cholesky factor).

A Gaussian is a point and an ellipse.

Next: **Maximum Likelihood Estimation** — we can now describe data with distributions; next we *fit* them.